

3.3 – Docker on Pi (ARM64)

Running Your Edge AI App Inside a Container

EdgeAI Engineering Bootcamp — Analog Data

What You Will Learn

By the end of this session you will be able to:

- Explain what Docker is and why it is useful on a Raspberry Pi
 - Install Docker correctly on Raspberry Pi OS Bookworm (ARM64)
 - Understand images, containers, and the difference between them
 - Write a multi-stage `Dockerfile` that builds a small, production-ready image
 - Write a `docker-compose.yml` that runs multiple services together
 - Set memory and CPU resource limits to protect the Pi from being overwhelmed
 - Manage containers using essential Docker commands
-

What You Will Build

By the end of this session you will have:

- A Docker image of your Edge AI inference app built for ARM64
 - A `docker-compose.yml` that runs four services together: inference engine, MQTT broker, FastAPI server, and SQLite storage
 - Memory and CPU limits set on each container so the Pi never runs out of resources
-

Part 1 — What Is Docker and Why Do We Need It?

The "It Works on My Machine" Problem

Imagine you build a Python inference app on your laptop. It works perfectly. You copy it to the Raspberry Pi and run it — and it immediately crashes with errors like:

```
ModuleNotFoundError: No module named 'tflite_runtime'  
ImportError: libgomp.so.1: cannot open shared object file
```

Why? Because your laptop is running x86-64 with Python 3.11 and certain library versions, but the Pi is running ARM64 with Python 3.9 and different library versions. The environment is completely different.

You could spend hours manually installing the right versions of everything on the Pi. Then you do it again for the second Pi. And again for the third. And six months later a system update changes something and everything breaks again.

Docker solves this. It packages your code AND everything it needs (Python version, libraries, system dependencies, config files) into one portable unit called a **container**. The container runs identically on your laptop, on a Pi 4B, on a Pi 5, and on a cloud server — because it carries its own environment with it.

The Real-World Analogy

Think of a container like a **shipping container** on a cargo ship.

Before shipping containers existed, cargo was loaded piece by piece. Every port used different methods, cranes, trolleys — it was chaos. A crate packed in India might not fit the unloading equipment in Singapore.

Shipping containers solved this: pack everything into a standard box. Every port in the world can handle that box the same way. What's inside doesn't matter — the outside is always the same standard shape.

Docker containers work the same way. Pack your app and all its dependencies into a standard container. Every machine with Docker installed can run it the same way.

Key Concepts — Before Writing Any Code

You must understand these four terms clearly before touching Docker:

Image

A Docker **image** is a read-only template — a complete snapshot of an environment that includes the OS layer, runtime, libraries, and your code. Think of it as a recipe or a blueprint.

An image is built once and can be used to create many containers.

Container

A **container** is a running instance of an image. When you start an image, Docker creates a container from it. The container is the actual running process.

The relationship: **Image is the recipe. Container is the dish.**

You can run ten containers from the same image simultaneously — just like baking ten cakes from the same recipe.

Dockerfile

A `Dockerfile` is a text file with step-by-step instructions for building an image. Docker reads the `Dockerfile` and executes each instruction to produce the final image.

Docker Compose

`docker-compose.yml` is a configuration file that defines and runs multiple containers together as one application. Instead of starting each container with a long `docker run` command, you describe everything in one file and start it all with one command: `docker compose up`.

ARM64 — Why It Matters on a Raspberry Pi

The Raspberry Pi uses an **ARM64** processor (also called `armv8`). Most computers use **x86-64** (also called `amd64`). These are completely different CPU architectures.

A Docker image built for x86-64 will NOT run on a Raspberry Pi. You need images built specifically for **ARM64** (or multi-architecture images that include both).

When pulling images from Docker Hub, always check for ARM64 support. When building your own images, build them directly on the Pi — the Pi builds them natively for ARM64.

Part 2 — Installing Docker on Raspberry Pi OS Bookworm

Do not install Docker from the Raspberry Pi OS package manager (`apt install docker.io`). That version is usually outdated. Always use the official Docker installation script.

Step-by-Step Installation

Step 1 — Update your system first

```
sudo apt update && sudo apt upgrade -y
```

Step 2 — Download and run the official Docker install script

```
curl -sSL https://get.docker.com | sh
```

This script automatically detects that you are on ARM64 Raspberry Pi OS and installs the correct version of Docker Engine.

Step 3 — Add your user to the docker group

By default, Docker commands require `sudo`. Adding your user to the `docker` group lets you run Docker commands without `sudo`:

```
sudo usermod -aG docker $USER
```

Step 4 — Apply the group change

You need to log out and back in, or run:

```
newgrp docker
```

Step 5 — Verify the installation

```
docker --version
# Docker version 26.x.x, build xxxxxxxx

docker run hello-world
```

If you see `Hello from Docker!` — Docker is installed and working correctly. 

Step 6 — Enable Docker to start at boot

```
sudo systemctl enable docker
sudo systemctl start docker
```

Step 7 — Install Docker Compose

Docker Compose is included with modern Docker Engine installations, but verify:

```
docker compose version
# Docker Compose version v2.x.x
```

If it is missing:

```
sudo apt install docker-compose-plugin -y
```

Part 3 — Essential Docker Commands

Before writing Dockerfiles, learn these commands. You will use them constantly.

Image commands

```
# List all images stored on your Pi
docker images

# Download an image from Docker Hub
docker pull python:3.11-slim

# Remove an image
docker rmi python:3.11-slim

# Build an image from a Dockerfile in the current directory
docker build -t edgeai:latest .

# Build and tag with a version
docker build -t edgeai:v1.0 .
```

Container commands

```
# Run a container (creates + starts in one command)
docker run edgeai:latest

# Run in the background (detached mode)
docker run -d edgeai:latest

# Run with a name (easier to reference later)
docker run -d --name inference edgeai:latest

# Run and immediately enter an interactive shell (great for debugging)
docker run -it python:3.11-slim bash

# List running containers
docker ps

# List ALL containers including stopped ones
docker ps -a

# Stop a running container
docker stop inference

# Start a stopped container
docker start inference

# Remove a stopped container
docker rm inference

# Stop and remove in one command
docker rm -f inference
```

Logs and debugging

```
# View container logs
docker logs inference

# Watch logs live
docker logs -f inference

# View last 50 lines
docker logs --tail 50 inference

# Open a shell inside a running container
docker exec -it inference bash

# Check resource usage (CPU, memory) of all running containers
docker stats
```

Cleanup commands

```
# Remove all stopped containers, unused images, unused networks
docker system prune
```

```
# Remove everything including unused images (more aggressive)
docker system prune -a

# Check total disk usage by Docker
docker system df
```

Part 4 — Writing a Dockerfile

A `Dockerfile` contains instructions Docker executes from top to bottom to build your image. Each instruction creates a **layer** in the image.

Basic Dockerfile instructions

Instruction	Purpose
FROM	Sets the base image to start from
WORKDIR	Sets the working directory inside the container
COPY	Copies files from your machine into the container
RUN	Runs a shell command during the build
ENV	Sets an environment variable
EXPOSE	Documents which port the container listens on
CMD	The command to run when the container starts
ENTRYPOINT	Like CMD but harder to override — sets the main executable

Your first basic Dockerfile

```
# Start from the official Python 3.11 slim image (ARM64 compatible)
FROM python:3.11-slim

# Set the working directory inside the container
WORKDIR /app

# Copy your requirements file first (see Part 5 for why)
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of your application code
COPY . .

# The command to run when the container starts
CMD ["python3", "inference.py"]
```

Build it:

```
docker build -t edgeai:latest .
```

Run it:

```
docker run edgeai:latest
```

This works, but it has a serious problem for production use on a Pi — the image is large because it contains everything used to build the app (compilers, header files, build tools). We only need those things during the build, not while running. This is where **multi-stage builds** come in.

Part 5 — Multi-Stage Dockerfile

The Problem with a Single-Stage Build

When you install packages like `tf-lite-runtime` or `numpy` on a Pi, `pip` often needs to compile C extensions from source — which requires build tools like `gcc`, `g++`, header files, and other large packages. These tools are needed for building but are completely useless at runtime.

A single-stage build image keeps all of these build tools in the final image, making it very large — sometimes 800MB or more for a simple Python app.

Multi-stage builds solve this: use a large, fully-equipped build stage to compile and install everything, then copy only the finished results into a second small, clean runtime stage. The final image contains only what is needed to run the app.

Multi-Stage Build — Concept

```
Stage 1: builder
├─ Full Python image with build tools
├─ Install gcc, g++, headers
├─ pip install all packages (compiles C extensions)
└─ Result: large environment with compiled packages

      ↓ copy only compiled packages ↓

Stage 2: runtime
├─ Small Python slim image (no build tools)
├─ Copy compiled packages from builder
└─ Final image: small and clean
```

Multi-Stage Dockerfile for Edge AI Inference

```
# _____
# STAGE 1: builder
```

```
# Large image with all build tools – used only during build
# -----

FROM python:3.11 AS builder

# Install system-level build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    libatlas-base-dev \
    libhdf5-dev \
    && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR /build

# Copy requirements and install
# Copying requirements.txt BEFORE the rest of the code means
# Docker can cache this layer – if requirements.txt hasn't changed,
# pip install is skipped on the next build. This saves a lot of time.
COPY requirements.txt .

RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# -----
# STAGE 2: runtime
# Small, clean image – this is what actually runs on the Pi
# -----

FROM python:3.11-slim AS runtime

# Install only the runtime system libraries (not build tools)
RUN apt-get update && apt-get install -y --no-install-recommends \
    libatlas3-base \
    libgomp1 \
    && rm -rf /var/lib/apt/lists/*

# Copy compiled Python packages from the builder stage
COPY --from=builder /install /usr/local

# Set the working directory
WORKDIR /app

# Copy application source code
COPY inference.py .
COPY models/ ./models/

# Set environment variables with sensible defaults
ENV MODEL_PATH=/app/models/model.tflite
ENV LOG_LEVEL=INFO
ENV CONFIDENCE_THRESHOLD=0.75

# Document that this app listens on port 8000
EXPOSE 8000

# Run as a non-root user for security
```



```
RUN useradd --create-home appuser
USER appuser

# Start the inference app
CMD ["python3", "inference.py"]
```

What `COPY --from=builder /install /usr/local` does:

This is the key multi-stage command. It copies the installed Python packages from the `builder` stage (stored at `/install`) into the `runtime` stage (at `/usr/local`, which is on Python's path). The `builder` stage itself is then discarded — it never becomes part of the final image.

Result: Image size comparison

Approach	Approximate image size
Single-stage (full Python image)	800MB – 1.2GB
Multi-stage (slim runtime)	120MB – 200MB

On a Raspberry Pi with a 16GB or 32GB SD card, this difference matters significantly — especially if you are running multiple containers.

The `requirements.txt` Caching Trick

Notice this pattern in the Dockerfile:

```
COPY requirements.txt .          # copy requirements FIRST
RUN pip install -r requirements.txt # install
COPY . .                        # THEN copy the rest of the code
```

Docker builds images in layers. Each instruction is a layer. If a layer has not changed since the last build, Docker reuses the cached version and skips re-running that instruction.

By copying `requirements.txt` before the rest of your code, the `pip install` layer is only rebuilt when `requirements.txt` actually changes. Changing a single line in `inference.py` does NOT trigger a new `pip install` — saving you several minutes on every build.

If you copied everything at once with `COPY . .` before `pip install`, every single code change would force a full reinstall of all packages.

Part 6 — `docker-compose.yml`

Why Docker Compose?

A real Edge AI system on a Pi is not just one Python script. It typically involves:

- An **inference engine** — your Python script running the model
- An **MQTT broker** — receives sensor data and distributes messages
- A **FastAPI server** — exposes inference results as an HTTP API
- A **SQLite or database** — stores historical readings and results

You could start each of these with a separate `docker run` command, but that is difficult to manage. You would need to remember four different commands with all their flags, ports, volumes, and environment variables.

`docker-compose.yml` defines all four services in one file. You start everything with:

```
docker compose up -d
```

And stop everything with:

```
docker compose down
```

Full `docker-compose.yml` — Inference + MQTT + FastAPI + SQLite

```
# docker-compose.yml
# Edge AI Stack — Analog Data EdgeAI Bootcamp

version: "3.9"

services:

# -----
# 1. MQTT Broker (Eclipse Mosquitto)
# Receives messages from sensors, distributes
# them to the inference engine
# -----
mqtt:
  image: eclipse-mosquitto:2.0
  container_name: mqtt_broker
  restart: always
  ports:
    - "1883:1883"      # MQTT protocol port
    - "9001:9001"      # WebSocket port (optional)
  volumes:
    - ./mosquitto/config:/mosquitto/config    # config files
    - ./mosquitto/data:/mosquitto/data        # persistent data
    - ./mosquitto/log:/mosquitto/log          # logs
  mem_limit: 64m      # MQTT broker is lightweight — 64MB is plenty
  cpus: "0.25"        # use at most 25% of one CPU core

# -----
# 2. Inference Engine (your Python app)
# Reads frames, runs the TFLite model,
# publishes results to MQTT
# -----
```

```

inference:
  build:
    context: .          # look for Dockerfile in current directory
    dockerfile: Dockerfile
  container_name: edgeai_inference
  restart: always
  depends_on:
    - mqtt              # wait for MQTT broker to start first
  environment:
    - MODEL_PATH=/app/models/model.tflite
    - MQTT_BROKER=mqtt  # use the service name – Docker resolves it to the
container IP
    - MQTT_PORT=1883
    - MQTT_TOPIC=sensors/inference
    - LOG_LEVEL=INFO
    - CONFIDENCE_THRESHOLD=0.75
  volumes:
    - ./models:/app/models          # mount model files from host
    - inference_data:/app/data      # persistent storage for results
  mem_limit: 512m                  # inference needs more memory for the model
  cpus: "1.5"                      # allow up to 1.5 CPU cores
  devices:
    - /dev/video0:/dev/video0      # pass camera through to container

# -----
# 3. FastAPI Server
# Exposes inference results as a REST API
# Other systems can query: GET /results/latest
# -----
api:
  build:
    context: ./api        # look for Dockerfile in ./api subfolder
    dockerfile: Dockerfile
  container_name: edgeai_api
  restart: always
  depends_on:
    - inference
    - mqtt
  ports:
    - "8000:8000"         # expose API on port 8000 of the Pi
  environment:
    - MQTT_BROKER=mqtt
    - DB_PATH=/app/db/results.db
    - API_KEY=${API_KEY}   # read from .env file on the host
  volumes:
    - inference_data:/app/db # shared volume with inference container
  mem_limit: 256m
  cpus: "0.5"

# -----
# 4. SQLite Backup Service
# Periodically backs up the SQLite database
# to a safe location
# -----
db_backup:

```

```

image: alpine:3.18
container_name: db_backup
restart: always
volumes:
  - inference_data:/data          # access the same data volume
  - ./backups:/backups           # write backups here on the host
command: >
  sh -c "while true; do
    cp /data/results.db /backups/results_$(date +%Y%m%d_%H%M).db;
    echo 'Backup complete';
    sleep 3600;
  done"
mem_limit: 32m
cpus: "0.1"

# -----
# Named volumes – Docker manages these
# Data persists even when containers are removed
# -----
volumes:
  inference_data:
    driver: local

```

Understanding the Key Compose Concepts

restart: always

Same as `Restart=always` in systemd. If the container exits for any reason, Docker restarts it automatically. Essential for production services.

depends_on:

```

depends_on:
  - mqtt

```

This tells Docker Compose to start the `mqtt` container before starting `inference`. Without this, `inference` might try to connect to the MQTT broker before it is ready.

Note: `depends_on` only waits for the container to START, not for the service inside it to be READY. For a proper health check, see the `healthcheck` section at the end of this module.

ports:

```

ports:
  - "8000:8000"

```

Format: `"host_port:container_port"`. Maps port 8000 of the Pi (host) to port 8000 inside the container. Without this, the port is only accessible from within Docker's internal network — not from your laptop or browser.

volumes:

```
volumes:
- ./models:/app/models
```

Format: `host_path:container_path` . Mounts a folder from the Pi's filesystem into the container. Changes made on either side are immediately visible on the other side.

This is how you:

- Provide the model file to the inference container without baking it into the image
- Persist data so it survives container restarts

environment:

```
environment:
- MQTT_BROKER=mqtt
- LOG_LEVEL=INFO
- API_KEY=${API_KEY}
```

Sets environment variables inside the container. `${API_KEY}` reads from a `.env` file in the same folder as `docker-compose.yml` .

Docker Compose automatically reads a file named `.env` in the same directory. Create it:

```
# .env - Docker Compose automatically loads this
API_KEY=your_secret_api_key
CONFIDENCE_THRESHOLD=0.75
```

Service names as hostnames

Inside a Docker Compose network, each service is reachable from other services using its **service name** as the hostname. So in the inference container, you connect to the MQTT broker with:

```
MQTT_BROKER = os.environ.get("MQTT_BROKER", "mqtt") # "mqtt" is the service name
```

Docker's internal DNS resolves `mqtt` to the correct container IP automatically. You never need to hardcode IP addresses.

Part 7 — Resource Limits: `mem_limit` and `cpus`

Why Resource Limits Are Critical on a Pi

A Raspberry Pi 4B has 1–8GB of RAM and 4 CPU cores. If one container goes rogue — say, a memory leak in the inference engine — it can consume all available RAM and crash the entire system, taking down the MQTT broker, the API, and the database with it.

Resource limits prevent this. Each container gets a maximum slice of RAM and CPU. If it tries to use more than its limit, Docker throttles or kills it — leaving enough resources for the other containers to keep

running.

mem_limit — Memory Limit

```
mem_limit: 512m    # maximum 512 megabytes of RAM
```

Accepted formats:

Value	Meaning
64m	64 megabytes
512m	512 megabytes
1g	1 gigabyte
2560000000	in bytes (not recommended — hard to read)

What happens when a container exceeds its memory limit?

Docker kills the container with an **OOM (Out Of Memory) kill**. Because `restart: always` is set, Docker immediately restarts it. The other containers continue running unaffected.

This is far better than the alternative — one container starving the entire Pi of memory and causing a kernel panic.

cpus — CPU Limit

```
cpus: "1.5"    # allow use of up to 1.5 CPU cores worth of compute
```

This does NOT pin the container to a specific CPU core. It limits how much total CPU time the container can consume across all cores.

Value	Meaning
"0.1"	10% of one CPU core
"0.5"	50% of one CPU core
"1.0"	100% of one CPU core — equivalent to one full core
"2.0"	Up to two full cores worth of CPU time
"4.0"	All four cores (Pi 4B maximum)

Recommended limits for a Pi 4B (4 cores, 4GB RAM):

Service	mem_limit	cpus	Reason
MQTT broker	64m	"0.25"	Very lightweight — just routes messages
Inference engine	512m	"1.5"	Needs headroom for model + frame buffers
FastAPI server	256m	"0.5"	Web server — moderate needs
DB backup	32m	"0.1"	Simple file copy — barely needs resources
Total	~864m	~2.35	Leaves ~3GB RAM and ~1.65 cores for the OS

Part 8 — Docker Compose Commands

```
# Start all services in the background
docker compose up -d

# Start all services and watch logs live
docker compose up

# Start only specific services
docker compose up -d mqtt inference

# Stop all services (containers are kept)
docker compose stop

# Stop and REMOVE all containers (data volumes kept)
docker compose down

# Stop, remove containers AND delete volumes (all data gone)
docker compose down -v

# Rebuild images and restart all services
docker compose up -d --build

# View logs of all services
docker compose logs

# Watch logs of a specific service live
docker compose logs -f inference

# Check the status of all services
docker compose ps

# Open a shell inside a running service container
docker compose exec inference bash

# See resource usage of all compose containers
docker stats
```

Part 9 — Useful Dockerfile Patterns for Pi

Checking the architecture inside a container

```
docker run --rm python:3.11-slim uname -m
# aarch64      ← this confirms ARM64
```

Keeping images small — best practices

```
# Combine RUN commands with && to reduce layers
RUN apt-get update \
    && apt-get install -y --no-install-recommends gcc libatlas-base-dev \
    && rm -rf /var/lib/apt/lists/*
```

Breaking `apt-get update`, `apt-get install`, and `rm -rf` into separate `RUN` instructions creates three layers. Combining them into one `RUN` with `&&` creates one layer — and crucially, the `rm -rf /var/lib/apt/lists/*` actually reduces the image size because it is in the same layer as the install.

.dockerignore — Exclude files from the build context

Create a `.dockerignore` file in the same folder as your `Dockerfile` :

```
# .dockerignore
.env
.git
__pycache__
*.pyc
*.egg-info
.venv
venv/
*.log
backups/
```

Docker sends all files in the build context to the Docker daemon before building. `.dockerignore` excludes files that should never be in the image — especially `.env` (which contains secrets) and `.git` (which can be very large).

Part 10 — Troubleshooting

Problem	Likely cause	Fix
Cannot connect to Docker daemon	Docker service not running	<code>sudo systemctl start docker</code>
permission denied while trying to connect	User not in docker group	<code>sudo usermod -aG docker \$USER</code> then log out and back in
no matching manifest for linux/arm64	Image has no ARM64 version	Find an ARM64-compatible image on Docker Hub

Problem	Likely cause	Fix
Container exits immediately	App crashed on startup	<code>docker logs container_name</code> to see the error
Port already in use	Another process using that port	Change the host port: <code>"8001:8000"</code>
<code>Cannot allocate memory</code>	Pi doesn't have enough free RAM	Reduce <code>mem_limit</code> on other containers or reduce number of containers
Volume changes not reflected	Forgot to mount the volume	Check <code>volumes:</code> section in compose file
Build is slow every time	Not using layer caching correctly	Copy <code>requirements.txt</code> before <code>COPY .</code>
OOM kill loop	<code>mem_limit</code> too low	Increase <code>mem_limit</code> for the affected container

Quick Reference Card

Dockerfile instructions

Instruction	Example
<code>FROM</code>	<code>FROM python:3.11-slim</code>
<code>WORKDIR</code>	<code>WORKDIR /app</code>
<code>COPY</code>	<code>COPY requirements.txt .</code>
<code>RUN</code>	<code>RUN pip install -r requirements.txt</code>
<code>ENV</code>	<code>ENV LOG_LEVEL=INFO</code>
<code>EXPOSE</code>	<code>EXPOSE 8000</code>
<code>CMD</code>	<code>CMD ["python3", "inference.py"]</code>
<code>COPY --from=</code>	<code>COPY --from=builder /install /usr/local</code>

Resource limits

Field	Example	Meaning
<code>mem_limit</code>	<code>512m</code>	Max 512MB RAM
<code>cpus</code>	<code>"1.5"</code>	Max 1.5 CPU cores

Docker Compose one-liners

Command	What it does
<code>docker compose up -d</code>	Start everything in background
<code>docker compose down</code>	Stop and remove containers
<code>docker compose logs -f inference</code>	Watch inference logs live
<code>docker compose ps</code>	Check status of all services
<code>docker compose exec inference bash</code>	Shell into inference container
<code>docker compose up -d --build</code>	Rebuild and restart

Summary Checklist

- ☐ I can explain what Docker is using the shipping container analogy
 - ☐ I can explain the difference between an image and a container
 - ☐ I know why ARM64 matters and how to check if an image supports it
 - ☐ I can install Docker on Raspberry Pi OS Bookworm using the official script
 - ☐ I know the essential `docker run`, `docker ps`, `docker logs`, `docker exec` commands
 - ☐ I can write a basic `Dockerfile` with `FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD`
 - ☐ I understand the `requirements.txt` caching trick and why it saves build time
 - ☐ I can write a multi-stage `Dockerfile` and explain what `COPY --from=builder` does
 - ☐ I can write a `docker-compose.yml` with multiple services, ports, volumes, and environment variables
 - ☐ I understand what `depends_on` does and its limitation
 - ☐ I can set `mem_limit` and `cpus` resource limits and explain why they matter on a Pi
 - ☐ I can use `docker compose up -d --build`, `down`, `logs -f`, `ps`, `exec`
 - ☐ I can troubleshoot a container that exits immediately using `docker logs`
-

Notes prepared by Analog Data — EdgeAI Engineering Bootcamp Copyright 2026 Analog Data. Strictly for enrolled students only.